C语言期末复习.

赋值表达式中，如 a=3，这个的返回值即为3.
a=0，返回值即为0

if(a=0) 下面是不执行的.

自加自减一定要独占一个表达式，否则未定义行为算不出来的东西的.

strlen这个不算\0，有n个字就是n.

指针与数组的本质公式:
a[i] = *(a+i) a为首字元素地址.

指针数组: int* p[3]；里面有3个元素，每个元素的类型均为int*.

数组指针 int (*p)[3] 这个是指向有3个元素数组的指针，可以把有3个元素的数组地址给它，用的时候(*p)[0]就是那个数组第一个元素了.

在二维数组中，写数组名会自动退化成首行的址.

结构体：定义：
```
Typedef struct Student {
        char name[20];
        int age;
        float score;
}Stu; -> 记住结构的定义需分号.
```
↓
你想简单叫的类型名，它就和 int, double 什么的是同一层级的东西了，不用再加 struct 了.

一般来说，结构的定义都在 main 函数外面.

结构体的初始化：Stu stu = {"Luo", 18, 97.0};
这是在为一个 Stu 类型叫做 stu 的变量做初始化，且必须一一对应.
"Luo"对应的是在上方声明的 char name[20]，依此类推.

结构体数组的初始化：Stu stu[2] = {{"Luo", 18, 97.0}, {"X", 19, 97.0}
↓ ↓
Stu[0] Stu[1].

结构体的调用：十分简单 printf("%s", Stu[0].name); => Luo
↓
字符数组名，是'L'的地址，
printf中的%s可以一下把整个全打出来.

结构体指针：用于在函数中影响值.

void swap(Stu *p){
 p->name = "hi"; 经典错误，除了初始化不能给数组直接赋值
 p->age = 3; 没问题.
                ↓
        可以 #include <string.h> 先
 strcpy(p->name, "hi"); 可以的.
}

在main中.
 Stu class[2] = {{"Luo", 18, 97.0}, {"X", 19, 99.3}};
 Stu *p = &class[0];
 swap(p);
这样就将 class[0] 中的 "Luo" 换成了 "hi"，18换成了3.

重要：不要把结构看成数组，它实际和int、float没有什么太大区别.

全大写　文件名后缀　模式　r：读 read　w：写 write　a 追加.

文件操作： FILE *fp

是一个定义好了 ↙ fp = fopen("h.txt", "w"); 会返回FILE指针

的结构体，不用管。

在r模式里如果没找到对应文件，将返回一个NULL给指针fp.

w会自动创建个新的，如果之前有这个文件，那就会清空了再写新的进去.

w：向内写东西，用fprintf，与printf极其相似，可以类比，不过这个是写进文件而printf是写到屏幕上. 格式：fprintf(fp, "Hello, world\n");

写好了以后 fclose(fp); 进行保存　注意：这些函数都是接收~~[illegible]~~地址fp.

fput系列： fputc(单个字符, fp) 写一个进去
　　　　　fputs(字符串, fp) 写字符串.

r：向外读东西 fgets(str, n, fp);
　你要存进的数组名 ↓ ↘ 来源.
　(首字母地址)　最多读n个(n-1，再留一个给\0)

fgets的终止：①遇见"\n" ②读到n-1了留一个\0的时候 ③文件~~终~~尾止，EOF.
　　↓
　也会存入数组.

fscanf(fp, "%d", &age); 除了前头有个数据来源，与scanf无差。

fgets 如果成功读取一行，那就把这行返给你写的那个接收数组里。
失败，则返回 NULL

∴ while (fgets(str, 20, fp) != NULL)
printf("%s", str)

读出一行会把 \n 也写进 str里，用上面这个循环可以把文件内内容全打到屏幕上。
在printf的时候也会打出来，所以不用额外 \n。

而 fgetc(fp) 读一个出来。
它会返回一个int值，所以你用来存的一定得是 int类型的变量，不能用char，
返回是ASCII值。因为EOF一般是-1。
写到屏幕上用 putchar，或直接%c都可行

光标移动. rewind(fp) 使光标回到文件开头。
fseek(fp, 2, SEEK_SET);
偏移量 从文件开始 => 从第一个移动到第三个上。

负值就是往回移. SEEK_CUR 当前光标位
SEEK_END 文件末尾

CamScanner

C语言期末复习　　　　　　再说白了就是一堆地址罢了.

↑

命令行　　　　　　　　这是一个指针为元素的数组.

↑

程序内写法：int main(int argc, char *argv[])

argc: argument count 个数

argv: argument vector 内容.

argv[0]里面是程序的名字.

易错的点：假如说我向命令行中输了字符串"hello"

那argv[1]是首字母'h'的地址

*argv[1]得到'h'

printf的时候"%s", argv[1]就可以把"hello"全输出在屏幕上.

多层指针：**p 指向指针的指针

比如说：int a=10;

int *p=&a;　　&p对指针求址，所以q类型是int**才能与之匹配

int **q=&p;　　到位. 此时这个q就是~~*p的~~ p的地址

↓

一个地址

注意：*q==p　※

**q=*p=a.

类比来看 ~~&(*p)~~ &(*p),则就得到p

*p对应a，所以对a求址自然得到p(其地址).

上强度了：你也可以在 int main(int argc, char** argv)

现在argv的类型是一个char**, 双层指针.

我们知道，写数组名会自动退化成首元素地址.

argv 为 ~~指针者~~ 首元素地址的地址，如果是字符串那就是首字符地址的地址.

*argv 首元素的地址

**argv 首元素.

但是如果你在后面加上[]就代表数组中的元素了(不能再当首元素地址用).

所以如果你不想用[]，只想操作地址 **(argv+1)就变成第二个元素啦！它与*argv[1]等价. 均代表第二个字符串的首字母. *(*argv+1)为首字符串的第二个字母.

⇕

*(argv[0]+1)

~~用好~~ 地址的地址，解引用，数组名是首元素地址，这样就搞懂啦！

CamScanner

在你的文件夹顶端输入cmd，然后dir你的C程序路径.
回车、"文件名.exe 空格 你的参数 (参数间要空格，输好了回车，你的程序就可以在命令行中执行.

动态内存分配

int *arr = (int*)malloc(5*sizeof(int));

⇓

将 malloc void*的返回值强制转成 int*，也是和前面

int*类型的arr对应.

此时我们便将5个 int内存大小的空间分配给了arr[]

注：因为数组名自动退化成首元素指针，所以有了动态内存分配

也可以反过来玩).

realloc重新调整

int *New_arr = ~~re~~ (int*)realloc(arr, 10*sizeof(int));

↓

将这个里有的元素原封不动，数组里容量
变成10个int，内容全扔进New-arr里.

用完了要free(arr)不然有可能内存泄漏，free早了后面又没法再用了.
注意free时机.